



Major Project, Online Shopping Website

Submitted By,

KOUSHIK MONDAL (32201219076)

MEGHNA HAZRA (32201219049)

MOUMITA DEY (32201219044)

MONODEEP BISWAS (32201219036)

CHANDRA SEKHAR KARMAKAR(32201219056)

(BACHELOR OF COMPUTER APPLICATION)

6th SEM

Under the Guidance of
Dr. Pintu Pal



Department of Computer Applications

Asansol Engineering College

Kalyanpur, Sen Raleigh Road, Asansol - 713304

CERTIFICATE

This is to certify that the project work entitled "*Online Shopping Website*" is a bonafide record of work carried out in the *Department of Computer Application*, Asansol Engineering College, Asansol.

Name	Roll Number	Registration Number
Koushik Mondal	32201219076	035732
Meghna Hazra	32201219049	036339
Moumita Dey	32201219044	035573
Chandra Sekhar Karmakar	32201219056	036258
Monodeep Biswas	32201219036	036886

The students of 6th Semester BCA 2021-22 under my / our supervision in requirement of partial fulfillment of the Award of Degree of BCA from Maulana Abul Kalam Azad University of Technology, West Bengal.

Signature of
The Project Guide
Name: Dr. Pintu Pal
Designation: Assistant Professor

Recommendation & Signature of
The Principal
Name: Dr. P. P. Bhattacharya

Recommendation & Signature of
The Head of Department
Name: Dr. D. K. PAL



Recommendation & Signature of
Internal / External Examiners

Abstract

In today's fast-changing business world, it's essential important to be able to respond to client needs in the most effective and timely manner. If your clients want to see your company online and have access to your products or services right away. Online Shopping is a lifestyle e-commerce website that sells a variety of fashion and lifestyle products (now Men's Wear).

This project allows registered users to browse the many products available and purchase desired things instantaneously, as well as place orders utilizing the Cash on Delivery (Pay Later) option. Administrators and managers can easily view orders placed using the Pay Later and Instant Pay options.

LITERATURE REVIEW

Python is an interpreted, high-level, general-purpose programming language. Created by Guido van Rossum and first released in 1991, Python's design philosophy emphasizes code readability with its notable use of significant whitespace. Its language constructs and object-oriented approach aims to help programmers write clear, logical code for small and large-scale projects. Python is dynamically typed and garbage-collected. It supports multiple programming paradigms, including procedural, object-oriented, and functional programming. Python is often described as a "batteries included" language due to its comprehensive standard library.

Django is a high-level Python Web framework that encourages rapid development and clean, pragmatic design. Built by experienced developers, it takes care of much of the hassle of Web development, so you can focus on writing your app without needing to reinvent the wheel. Django is a free and open-source web framework, which follows the model view-template architectural pattern. Django's primary goal is to ease the creation of complex, database-driven websites. The framework emphasizes reusability and "pluggability" of components, less code, low coupling, rapid development, and the principle of don't repeat yourself. Python is used throughout, even for settings files and data models. Django also provides an optional administrative create, read, update and delete interface that is generated dynamically through introspection and configured via admin models. Some well-known sites that use Django include the Public Broadcasting Service, Instagram, Mozilla, The Washington Times, Disqus, Bitbucket, and Next-door. It was used on Pinterest, but later the site moved to a framework built over Flask.

WEBSITE ARCHITECTURE FOR ECOMMERCE

Having a great site architecture means making products and categories findable on your website, in a way that users and search engines can reach them as efficiently as possible. There are two concepts regarding site architecture: Efficient crawling and indexing. This refers to the Technical Architecture or TA and Classifying, labelling and organizing content. This refers to Information Architecture or IA. Together, information and technical architecture form the site architecture (SA). A good understanding of these two concepts will help to build search engine optimized websites that are search-engine and user-friendly.

Information Architecture: is the process of classifying and organizing content on a website while providing user-friendly access to that content, via navigation. This process is done (or should be done) by information architects.

Technical architecture: is the process of designing the technical and functional aspects of a site. This is mostly done by web developers.

Proposed System

The proposed system aids in the creation of a website for the purchase and sale of products or goods via the internet. Unlike traditional commerce, which requires a person to travel to obtain items, our project will make it easier for customers to reduce physical labour and save time.

The application's primary principle is to allow users to shop virtually using the Internet and to purchase the things they want from the store. Customers do not need to go to stores to buy things under this planned method. Through the use of this system, a customer can order the goods that he or she want.

Through the use of this system, a customer can order the goods that he or she want. The system's administrator can be the shop owner. The shop owner can assign authorities to handle this specifically, who will assist the owner in managing clients and product orders.

Project Prerequisites and Features

Prerequisites to develop Online Shopping System in Django:

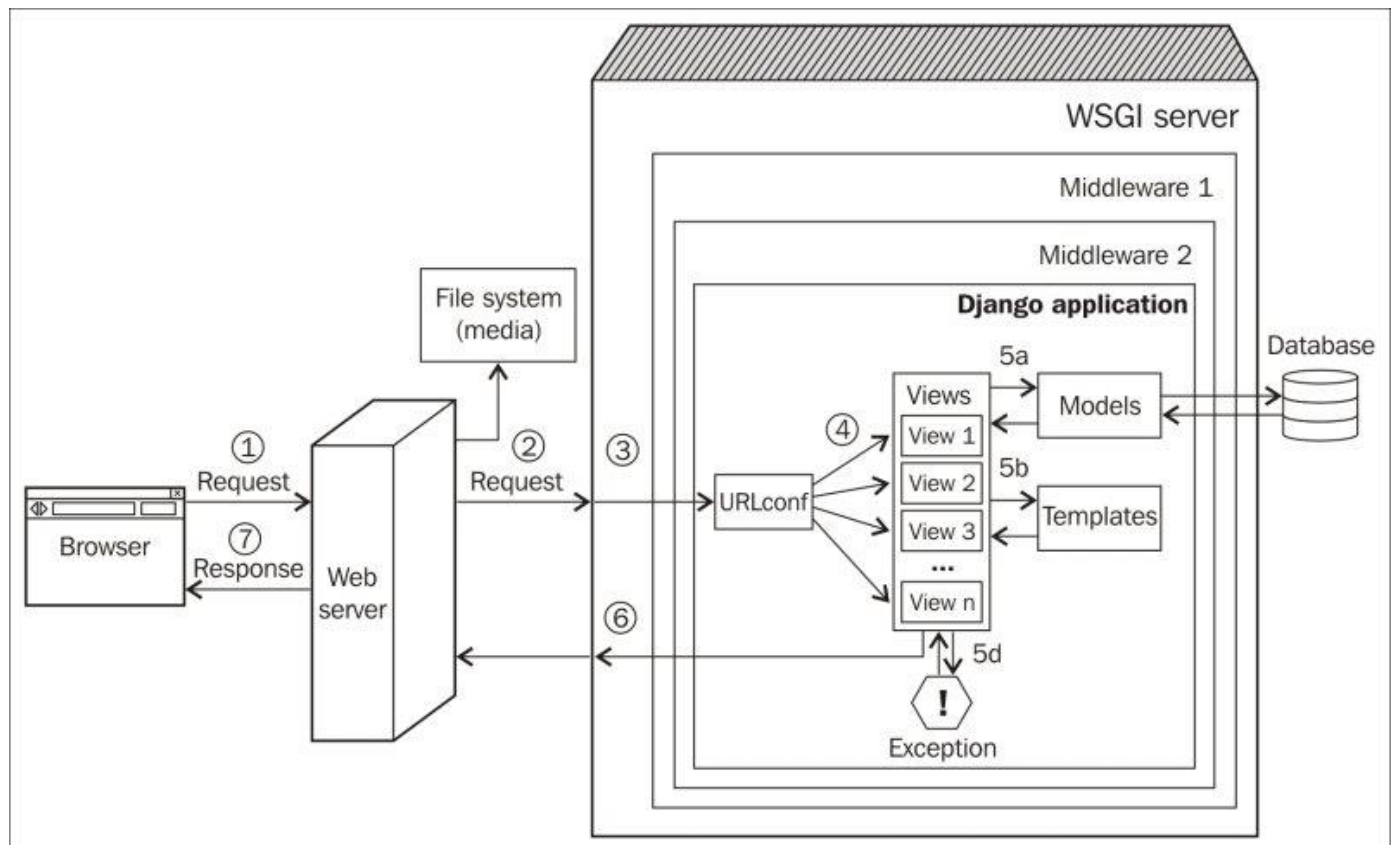
- ✓ Html
- ✓ CSS
- ✓ JavaScript
- ✓ Bootstrap
- ✓ Python Django.

Features:

- ✓ User Authentication (register and login)
- ✓ All Products with their view, key features and reviews by other users.
- ✓ Users are able to search for any product.
- ✓ Cart Functionality.
- ✓ Proper Order Summary Before Placing Order.
- ✓ Change Password Facility.
- ✓ Contact Us.

How does Django work?

To truly appreciate Django, we will need to peek under the hood and see the various moving parts inside.



The above figure shows the simplified journey of a web request from a visitor's browser to your Django application and back.

The numbered paths are as follows:

1. The browser sends the request (essentially, a string of bytes) to the web server.
2. The web server (say, Nginx) hands over the request to a WSGI server (say, uWSGI) or directly serves a file (say, a CSS file) from the filesystem.
3. Unlike a web server, WSGI servers can run Python applications. The request populates a Python dictionary called `environ` and, optionally, passes through several layers of middleware, ultimately reaching your Django application.
4. `URLconf` contained in the `urls.py` of your application selects a view to handle the request based on the requested URL. The request has turned into `HttpRequest` (a Python object).
5. The selected view typically does one or more of the following things:
 - a. Talks to a database via the models
 - b. Renders HTML or any other formatted response using templates
 - c. Returns a plain text response (not shown)

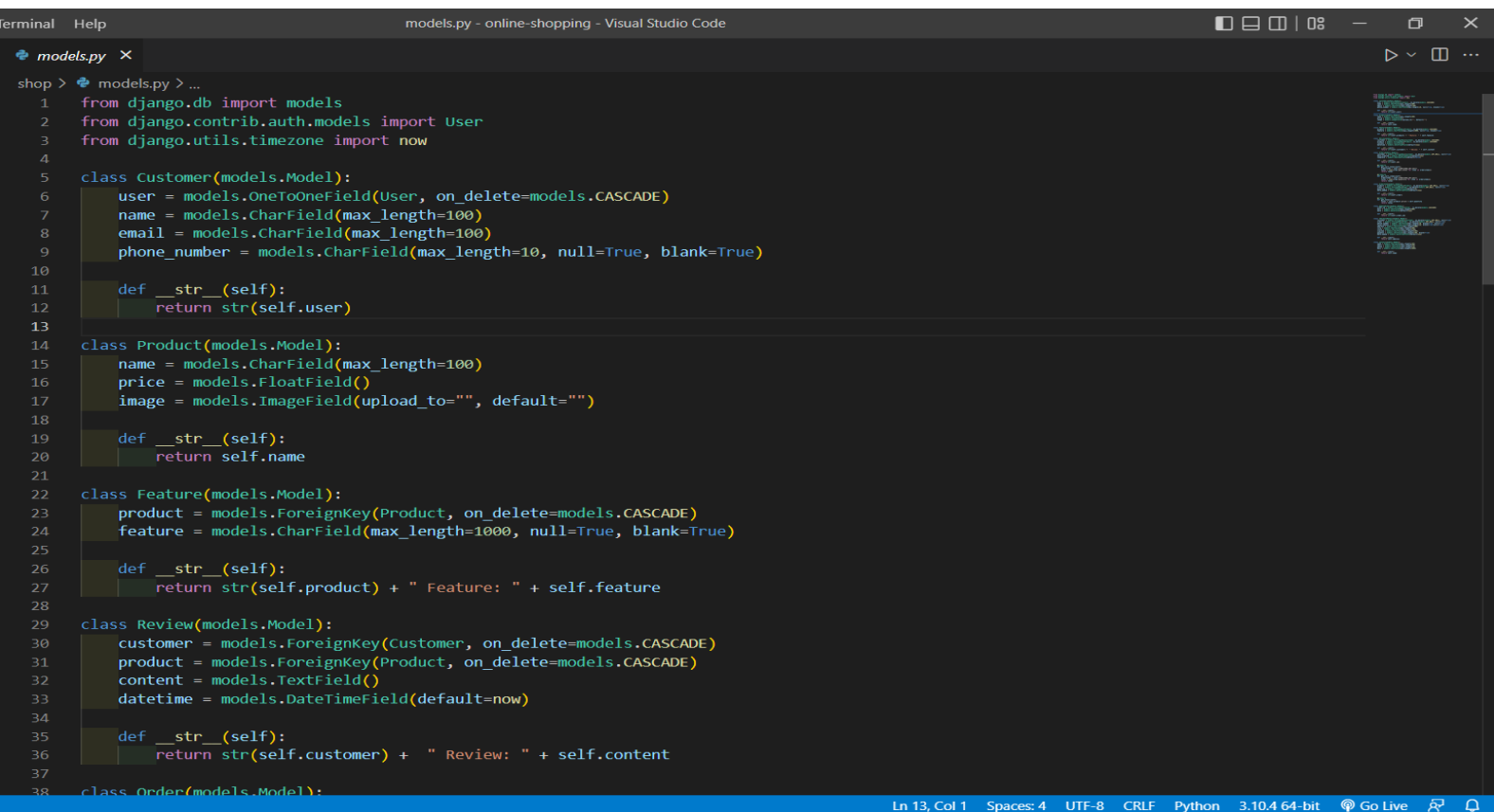
d. Raises an exception

- The Http Response object gets rendered into a string, as it leaves the Django application.
- A beautifully rendered web page is seen in your user's browser.

Project File Structure

Steps for Online Shopping System Python Project:

Models.py :



```
terminal  Help  models.py - online-shopping - Visual Studio Code

models.py x
shop > models.py > ...
1  from django.db import models
2  from django.contrib.auth.models import User
3  from django.utils.timezone import now
4
5  class Customer(models.Model):
6      user = models.OneToOneField(User, on_delete=models.CASCADE)
7      name = models.CharField(max_length=100)
8      email = models.CharField(max_length=100)
9      phone_number = models.CharField(max_length=10, null=True, blank=True)
10
11     def __str__(self):
12         return str(self.user)
13
14     class Product(models.Model):
15         name = models.CharField(max_length=100)
16         price = models.FloatField()
17         image = models.ImageField(upload_to="", default="")
18
19         def __str__(self):
20             return self.name
21
22     class Feature(models.Model):
23         product = models.ForeignKey(Product, on_delete=models.CASCADE)
24         feature = models.CharField(max_length=1000, null=True, blank=True)
25
26         def __str__(self):
27             return str(self.product) + " Feature: " + self.feature
28
29     class Review(models.Model):
30         customer = models.ForeignKey(Customer, on_delete=models.CASCADE)
31         product = models.ForeignKey(Product, on_delete=models.CASCADE)
32         content = models.TextField()
33         datetime = models.DateTimeField(default=now)
34
35         def __str__(self):
36             return str(self.customer) + " Review: " + self.content
37
38     class Order(models.Model):
```

```
terminal Help models.py - online-shopping - Visual Studio Code
models.py x
shop > models.py > ...
38 class Order(models.Model):
39     customer = models.ForeignKey(Customer, on_delete=models.SET_NULL, null=True)
40     date_ordered = models.DateTimeField(default=now)
41     complete = models.BooleanField(default=False)
42
43     def __str__(self):
44         return str(self.id)
45
46     @property
47     def get_cart_total(self):
48         orderitems = self.orderitem_set.all()
49         total = sum([item.get_total for item in orderitems])
50         return total
51
52     @property
53     def get_cart_items(self):
54         orderitems = self.orderitem_set.all()
55         total = sum([item.quantity for item in orderitems])
56         return total
57
58 class OrderItem(models.Model):
59     product = models.ForeignKey(Product, on_delete=models.SET_NULL, null=True)
60     order = models.ForeignKey(Order, on_delete=models.SET_NULL, null=True)
61     quantity = models.IntegerField(default=0)
62     date_added = models.DateTimeField(default=now)
63
64     def __str__(self):
65         return str(self.order)
66
67     @property
68     def get_total(self):
69         total = self.product.price * self.quantity
70         return total
71
72 class UpdateOrder(models.Model):
73     order_id = models.ForeignKey(Order, on_delete=models.CASCADE)
74     desc = models.CharField(max_length=500)
75     date = models.DateTimeField(default=now)
76
77     def __str__(self):
78         return str(self.order_id)
79
80 class CheckoutDetail(models.Model):
81     customer = models.ForeignKey(Customer, on_delete=models.SET_NULL, null=True)
82     order = models.ForeignKey(Order, on_delete=models.SET_NULL, null=True)
83     phone_number = models.CharField(max_length=10, blank=True, null=True)
84     total_amount = models.CharField(max_length=10, blank=True, null=True)
85     address = models.CharField(max_length=300)
86     city = models.CharField(max_length=100)
87     state = models.CharField(max_length=100)
88     zipcode = models.CharField(max_length=100)
89     payment = models.CharField(max_length=100, blank=True)
90     date_added = models.DateTimeField(default=now)
91
92     def __str__(self):
93         return self.address
94
95 class Contact(models.Model):
96     name = models.CharField(max_length=100)
97     email = models.CharField(max_length=50)
98     phone = models.CharField(max_length=10)
99     desc = models.CharField(max_length=1000)
100
101     def __str__(self):
102         return self.name
```

```
terminal Help models.py - online-shopping - Visual Studio Code
models.py x
shop > models.py > ...
71
72 class UpdateOrder(models.Model):
73     order_id = models.ForeignKey(Order, on_delete=models.CASCADE)
74     desc = models.CharField(max_length=500)
75     date = models.DateTimeField(default=now)
76
77     def __str__(self):
78         return str(self.order_id)
79
80 class CheckoutDetail(models.Model):
81     customer = models.ForeignKey(Customer, on_delete=models.SET_NULL, null=True)
82     order = models.ForeignKey(Order, on_delete=models.SET_NULL, null=True)
83     phone_number = models.CharField(max_length=10, blank=True, null=True)
84     total_amount = models.CharField(max_length=10, blank=True, null=True)
85     address = models.CharField(max_length=300)
86     city = models.CharField(max_length=100)
87     state = models.CharField(max_length=100)
88     zipcode = models.CharField(max_length=100)
89     payment = models.CharField(max_length=100, blank=True)
90     date_added = models.DateTimeField(default=now)
91
92     def __str__(self):
93         return self.address
94
95 class Contact(models.Model):
96     name = models.CharField(max_length=100)
97     email = models.CharField(max_length=50)
98     phone = models.CharField(max_length=10)
99     desc = models.CharField(max_length=1000)
100
101     def __str__(self):
102         return self.name
```

Code Explanation:

So, there are a total of 9 models created for this project.

1.Customer model:

It saves the basic data of the customers when they register themselves.

2.Product model:

It saves the data of the product. The admin can add a new product very easily using this model.

3.Feature model:

The admin can select the product and write any features about it. And all the features of that product will be visible to the users when they view a specific product.

4.Review model:

All the customers can write a review about a product which the customers can read before buying it.

5.Order model:

It stores the order details about the customer, mainly the order id.

6.OrderItems model:

It stores the order id of the customer from the order model and the products with their quantity.

7.Checkout Details model:

It stores mainly the exact address where the order is to be delivered.

8.Contact Model:

It stores the contact details of the user.

9.UpdateOrder Model

Create superuser:

After creating the models, we need to go to the admin panel to access the created models. Hence, we need a superuser who can access the models from the admin panel. The superuser can make any changes inside the models.

Creating superuser :

For creating the superuser use the following command:

```
python manage.py createsuperuser
```

Then, you can give the username, email and password.

Urls.py file


```
Terminal Help urls.py - online-shopping - Visual Studio Code
shop > urls.py > ...
1 from django.urls import path
2 from . import views
3
4 urlpatterns = [
5     path("", views.index, name="index"),
6     path("cart/", views.cart, name="cart"),
7     path("checkout/", views.checkout, name="checkout"),
8     path("update_item/", views.updateItem, name="update_item"),
9     path("product_view/<int:myid>/", views.product_view, name="product_view"),
10    path("search/", views.search, name="search"),
11    path("tracker/", views.tracker, name="tracker"),
12    path("contact/", views.contact, name="contact"),
13    path("loggedin_contact/", views.loggedin_contact, name="loggedin_contact"),
14
15    path("register/", views.register, name="register"),
16    path("change_password/", views.change_password, name="change_password"),
17    path("login/", views.Login, name="login"),
18    path("logout/", views.Logout, name="logout"),
19 ]
```

Code Explanation:

Above mentioned are the app urls. Hence, create a file for the urls inside the app. The last three urls are for the user authentication that is register, login and logout. The other urls consist of the entire project.

1.Home Page (index.html):

```
Terminal Help index.html - online-shopping - Visual Studio Code
shop > templates > index.html > script
1 {% block title of online shopping %} endblock %}
2 {% block css %}
3 {% endblock %}
4 {% block body %}
5
6 <div class="container mt-2 mb-3">
7
8     <div class="row">
9         {% for product in products %}
10            <div class="col-lg-4 my-4">
11                <div class="card shadow align-items-center" style="width: 20rem; height: 25rem;">
12                    
13                    <div class="card-body">
14                        <h5 class="card-title">{{product.name}}</h5>
15                        <hr>
16                        {% if request.user.is_authenticated %}
17                        <button data-product="{{product.id}}" data-action="add" class="btn add-btn update-cart"
18                            style="background-color: #8c5d4f; color: white;">Add To Cart</button>
19                        <a href="/product_view/{{product.id}}/" class="btn btn-outline-secondary">View</a>
20                        {% else %}
21                        <button class="btn" style="background-color: #8c5d4f; color: white;">Login to add the item</button>
22                        {% endif %}
23                        <h4 style="display: inline-block; float: right;">&nbsp;&{{product.price}}</h4>
24                    </div>
25                </div>
26            </div>
27        {% endfor %}
28    </div>
29    {% block js %}
30    <script>
31        {% if alert %}
32            alert("Logout Sucessfull.")
33            window.location.href = '/'
34        {% endif %}
35    </script>
36    {% endblock %}
37
```

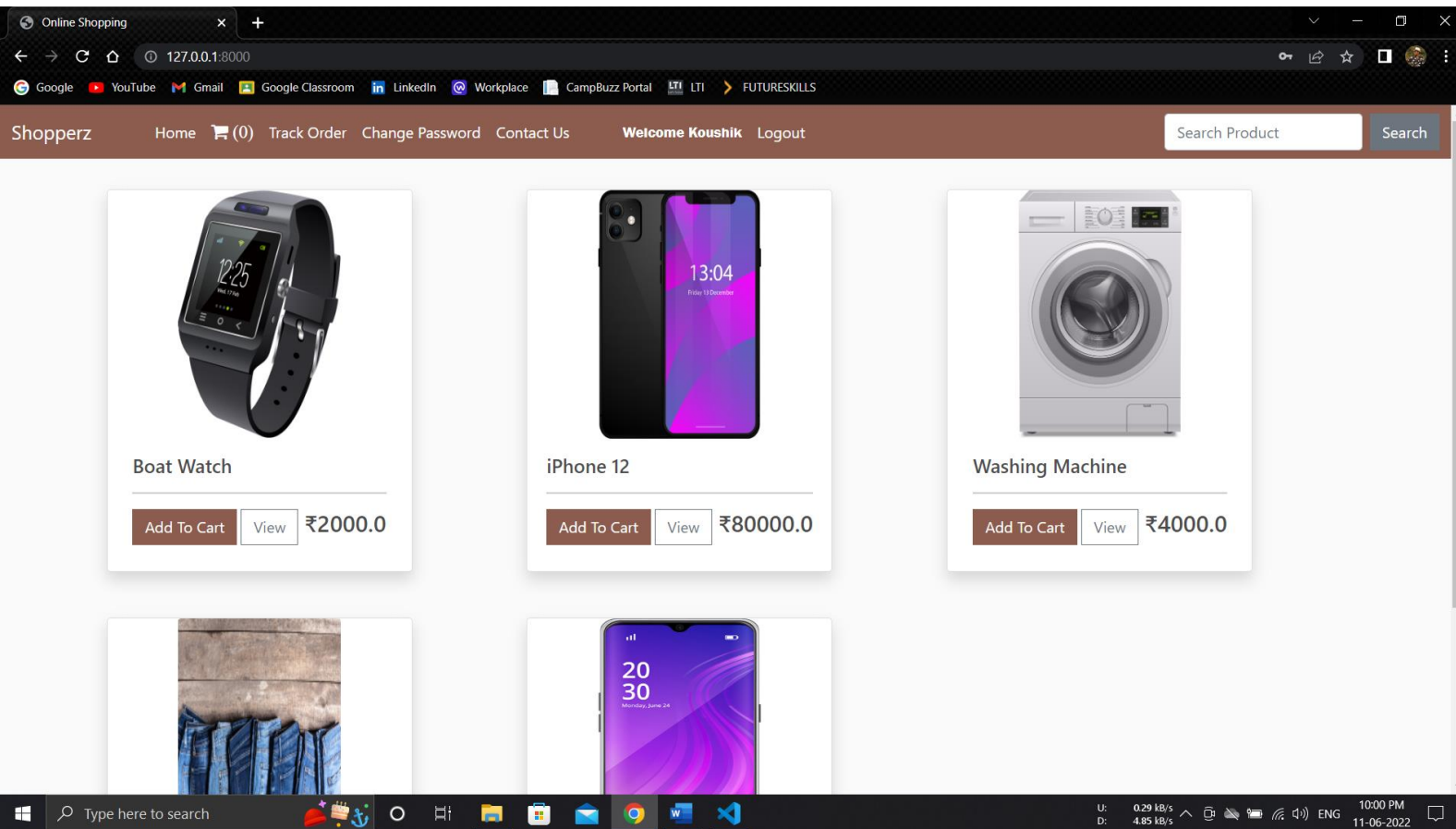
Views.py:

```

10 def index(request):
11     data = cartData(request)
12     items = data['items']
13     order = data['order']
14     cartItems = data['cartItems']
15
16     products = Product.objects.all()
17     return render(request, "index.html", {'products':products, 'cartItems':cartItems})
18

```

Home page with the customer logged in:



Code Explanation:

From the Product model, using a for loop all the products are displayed on the home screen with the image, name and price of the products. Two buttons are also created, one is for adding the item inside the cart and the other is to view the product. If the customers are not logged in then they are not able to add the item inside their cart. They can just see all the products. On the add to cart button, login to add the item will be written.

2. Search Product functionality (search.html):

```
terminal  Help  search.html - online-shopping - Visual Studio Code

< search.html 1 x

shop > templates > < search.html > < div.container.mt-2.mb-3 > < div.row > < div.col-lg-4.my-4 > < div.card.shadow.align-items-center > < div.card-body > < h4
1  {% extends 'base.html' %}
2
3  {% block title %} Searched Result {% endblock %}
4  {% block css %}
5  {% endblock %}
6  {% block body %}
7  {% if search %}
8
9  <div class="container mt-2 mb-3">
10
11      <div class="row">
12          {% for product in products %}
13              <div class="col-lg-4 my-4">
14                  <div class="card shadow align-items-center" style="width: 20rem; height: 23rem;">
15                      
16                      <div class="card-body">
17                          <h5 class="card-title">{{product.name}}</h5>
18                          <hr>
19                          {% if request.user.is_authenticated %}
20                              <button data-product="{{product.id}}" data-action="add" class="btn add-btn update-cart" style="background-color: #8c5d4f;">
21                              <a href="/product_view/{{product.id}}/" class="btn btn-outline-success">View</a>
22                          {% else %}
23                              <button class="btn" style="background-color: #8c5d4f; color: white;">Login to add the item</button>
24                          {% endif %}
25                          <h4 style="display: inline-block; float: right;">₹{{product.price}}</h4>
26                      </div>
27                  </div>
28              </div>
29          {% endfor %}
30      </div>
31  </div>
32  {% else %}
33  <h1>You forgot to search</h1>
34  {% endif %}
35  {% endblock %}
```

Views.py:

```
119
120 def search(request):
121     data = cartData(request)
122     items = data['items']
123     order = data['order']
124     cartItems = data['cartItems']
125     if request.method == "POST":
126         search = request.POST['search']
127         products = Product.objects.filter(name__contains=search)
128         return render(request, "search.html", {'search':search, 'products':products, 'cartItems':cartItems})
129     else:
130         return render(request, "search.html")
131
```

Code Explanation:

The customers can directly search for the product that they want on the search option given on the navigation bar. Whatever the customers write on the search bar will be sent through a post request and then saved inside a variable.

Afterwards, from the Product model the product name is filtered to the search value. The result is then displayed in the same cards as on the home page.

3. Product View Page (product_view.html):

```
Terminal  Help  product_view.html - online-shopping - Visual Studio Code

<> product_view.html X
shop > templates > <> product_view.html > ...
1  {% extends 'base.html' %}
2  {% load static %}
3  {% block title %} {{product.name}} {% endblock %}
4
5  {% block css %}
6  {% endblock %}
7  {% block body %}
8  {% load humanize %}
9  <div class="container mt-4">
10     <div class="row">
11         <div class="col-md-4">
12             <div class="row">
13                 
15             <br>
16             <div class="row">
17                 <button data-product="{{product.id}}" data-action="add" class="btn add-btn update-cart"
18                     style="background-color: #8c5d4f; color: white; width: 15rem;">Add To Cart</button>
19             </div>
20         </div>
21         <div class="col-md-8">
22             <h3>{{product.name}}</h3>
23             <p style="font-size: 25px;"><b>₹ {{product.price}}</b></p>
24             <h5>Key Features:</h5>
25             <ul>
26                 {% for i in feature %}
27                 <li>{{i.feature}}</li>
28                 {% endfor %}
29             </ul>
30             <br>
31             <h2>Add Reviews Here</h2>
32             <form method="POST" action="/product_view/{{product.id}}/"> {% csrf_token %}
33                 <div class="form-floating">
34                     <textarea class="form-control" placeholder="Leave a comment here" id="floatingTextarea2"
35                         style="height: 100px" id="content" name="content"></textarea>
36                     <label for="floatingTextarea2">Leave a feedback about the {{product.name}} here.</label>
37                 <br>
38                 <button type="submit" class="btn btn-primary">Add Review</button>
39             </form>
40         </div>
41     </div>
42 </div>
43 <div class="container mt-2">
44     <h2>All Reviews ({{reviews.count}})</h2>
45     {% for review in reviews %}
46     <div class="card">
47         <div class="card-header">
48             <span style="font-size: 22px;">{{review.customer}} </span> <span
49                 style="font-size: 12px;">{{review.datetime | naturaltime}}</span>
50         </div>
51         <div class="card-body">
52             <h6 class="card-text">{{review.content}}</h6>
53         </div>
54     </div>
55     <br>
56     {% endfor %}
57 </div>
58 </div>
59 </div>
60 {% endblock %}
61
62
63
64
65
```

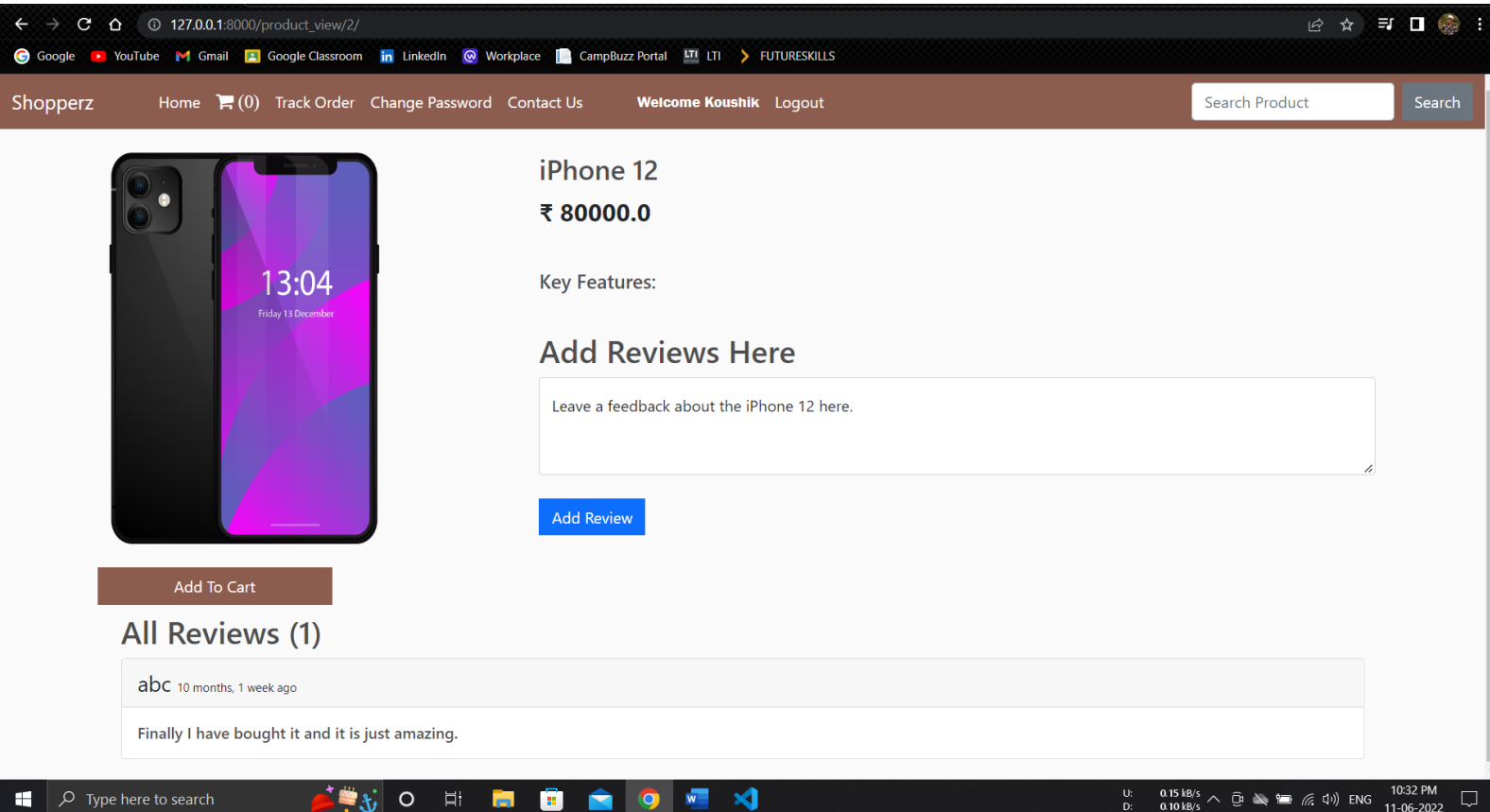
```
Terminal  Help  product_view.html - online-shopping - Visual Studio Code

<> product_view.html X
shop > templates > <> product_view.html > ...
34     <textarea class="form-control" placeholder="Leave a comment here" id="floatingTextarea2"
35         style="height: 100px" id="content" name="content"></textarea>
36     <label for="floatingTextarea2">Leave a feedback about the {{product.name}} here.</label>
37     <br>
38     <button type="submit" class="btn btn-primary">Add Review</button>
39 </div>
40 </form>
41 </div>
42 </div>
43 <div class="container mt-2">
44     <h2>All Reviews ({{reviews.count}})</h2>
45     {% for review in reviews %}
46     <div class="card">
47         <div class="card-header">
48             <span style="font-size: 22px;">{{review.customer}} </span> <span
49                 style="font-size: 12px;">{{review.datetime | naturaltime}}</span>
50         </div>
51         <div class="card-body">
52             <h6 class="card-text">{{review.content}}</h6>
53         </div>
54     </div>
55     <br>
56     {% endfor %}
57 </div>
58 </div>
59 </div>
60 </div>
61 {% endblock %}
62
63
64
65
```

Views.py:

```
103
104 def product_view(request, myid):
105     product = Product.objects.filter(id=myid).first()
106     feature = Feature.objects.filter(product=product)
107     reviews = Review.objects.filter(product=product)
108     data = cartData(request)
109     items = data['items']
110     order = data['order']
111     cartItems = data['cartItems']
112
113     if request.method=="POST":
114         content = request.POST['content']
115         review = Review(customer=customer, content=content, product=product)
116         review.save()
117         return redirect(f"/product_view/{product.id}")
118     return render(request, "product_view.html", {'product':product, 'cartItems':cartItems, 'feature':feature, 'reviews':reviews})
119
```

Product View:



Code Explanation:

After clicking on the view button, the customers can view the specific product with their key features and reviews. After reading the key features and the reviews about the product, the customer can buy the product by clicking on the add to cart button. The customer who has brought that specific product can write their review about the product which the other customer is able to read.

4. Cart Page (cart.html):

```
terminal  Help  carthtml - online-shopping - Visual Studio Code

< carthtml x

shop > templates > < carthtml > ...
1  {% extends 'base.html' %}
2  {% load static %}
3  {% block title %} Shopping Cart {% endblock %}
4
5  {% block css %}
6  {% endblock %}
7  {% block body %}
8  <div class="container-fluid row">
9      <div class="col-lg-12">
10         <div class="box-element">
11             <a href="/" class="btn btn-outline-dark">&#x2190 Continue Shopping</a>
12             <br><br>
13             <table class="table">
14                 <tr>
15                     <th>
16                         <h5>Items: <strong>{{order.get_cart_items}}</strong></h5>
17                     </th>
18                     <th>
19                         <h5>Total: <strong>₹{{order.get_cart_total}}</strong></h5>
20                     </th>
21                     <th><a href="/checkout/" class="btn btn-success" style="float: right; margin: 5px;">Checkout</a>
22                     </th>
23                 </tr>
24             </table>
25         </div>
26     </div>
27     <div class="box-element">
28         <div class="cart-row">
29             <div style="flex: 2;"><strong>Image</strong></div>
30             <div style="flex: 2;"><strong>Item</strong></div>
31             <div style="flex: 1;"><strong>Price</strong></div>
32             <div style="flex: 1;"><strong>Quantity</strong></div>
33             <div style="flex: 1;"><strong>Total</strong></div>
34         </div>
35         {% for item in items %}
36         <div class="cart-row">
37             <div style="flex: 2;"></div>
38             <div style="flex: 2;">{{item.product.name}}</div>
39             <div style="flex: 1;">{{item.product.price}}</div>
40             <div style="flex: 1;">{{item.quantity}}</div>
41             <div style="flex: 1;">{{item.get_total}}</div>
42             <div style="flex: 1;">
43                 
44                 
45             </div>
46         </div>
47         {% endfor %}
48     </div>
49 </div>
50 </div>
51 </div>
52 </div>
53 {% endblock %}
```

```
Terminal  Help  carthtml - online-shopping - Visual Studio Code

< carthtml x

shop > templates > < carthtml > ...
30 <div style="flex: 2;"><strong>Item</strong></div>
31 <div style="flex: 1;"><strong>Price</strong></div>
32 <div style="flex: 1;"><strong>Quantity</strong></div>
33 <div style="flex: 1;"><strong>Total</strong></div>
34 </div>
35 {% for item in items %}
36 <div class="cart-row">
37     <div style="flex: 2;"></div>
38     <div style="flex: 2;">{{item.product.name}}</div>
39     <div style="flex: 1;">{{item.product.price}}</div>
40     <div style="flex: 1;">{{item.quantity}}</div>
41     <div style="flex: 1;">{{item.get_total}}</div>
42     <div style="flex: 1;">
43         
44         
45     </div>
46 </div>
47 </div>
48 </div>
49 </div>
50 </div>
51 </div>
52 </div>
53 {% endblock %}
```


Views.py:

```
18
19 def cart(request):
20     data = cartData(request)
21     items = data['items']
22     order = data['order']
23     cartItems = data['cartItems']
24     try:
25         cart = json.loads(request.COOKIES['cart'])
26     except:
27         cart = {}
28     print('Cart:', cart)
29
30     for i in cart:
31         try:
32             cartItems += cart[i]["quantity"]
33
34             product = Product.objects.get(id=i)
35             total = (product.price * cart[i]["quantity"])
36
37             order["get_cart_total"] += total
38             order["get_cart_items"] += cart[i]["quantity"]
39
40             item = {
41                 'product':{
42                     'id':product.id,
43                     'name':product.name,
44                     'price':product.price,
45                     'image':product.image,
46                 },
47                 'quantity':cart[i]["quantity"],
48                 'get_total':total
49             }
50             items.append(item)
51         except:
52             pass
53     return render(request, "cart.html", {'items':items, 'order':order, 'cartItems':cartItems})
54
```

Cart Page:

← Continue Shopping

Items: 2 Total: ₹81000.0 [Checkout](#)

Image	Item	Price	Quantity	Total
	iPhone 12	₹80000.0	1	₹80000.0
	Jeans	₹1000.0	1	₹1000.0

Code Explanation:

By clicking on the shopping cart icon on the navigation bar, customers can see all the added items in the cart. The users can then increase or decrease the quantity of the products according to their requirements.

5. Checkout Page (checkout.html):

```
Terminal  Help  checkout.html - online-shopping - Visual Studio Code

> checkout.html 6 X
shop > templates > > checkout.html > div.container-fluid.row > div.col-lg-12 > div.box-element > div.cart-row > div > img.row-image
1  {% extends 'base.html' %}
2  {% load static %}
3  {% block title %} Checkout {% endblock %}
4  {% block css %}
5  {% endblock %}
6  {% block body %}
7  <div class="container-fluid row">
8  <div class="col-lg-12">
9      <div class="box-element">
10         <a class="btn btn-outline-dark" href="/cart/">&#x2190; Back to Cart</a>
11         <hr>
12         <h3>Order Summary</h3>
13         <hr>
14         {% for item in items %}
15         <div class="cart-row">
16             <div style="flex:2"></div>
17             <div style="flex:2">
18                 <p>{{item.product.name}}</p>
19             </div>
20             <div style="flex:1">
21                 <p>₹{{item.product.price}}</p>
22             </div>
23             <div style="flex:1">
24                 <p>x{{item.quantity}}</p>
25             </div>
26         </div>
27         {% endfor %}
28         <h5>Items: {{order.get_cart_items}}</h5>
29         <h5>Total: ₹{{order.get_cart_total}}</h5>
30     </div>
31 </div>
32 <div class="col-lg-12">
33     <div class="box-element" id="form-wrapper">
34         <form method="POST"> {% csrf_token %}
35         <div id="shipping-info">
36             <hr>
37             <h4>Checkout Details:</h4>
38             <hr>
39             <div class="">
40                 <input class="form-control" type="text" name="address" placeholder="Address.." required>
41             </div>
42             <div class="form-field mt-3">
43                 <input class="form-control" type="text" name="city" placeholder="City.." required>
44             </div>
45             <div class="form-field">
46                 <input class="form-control" type="text" name="state" placeholder="State.." required>
47             </div>
48             <div class="form-field">
49                 <input class="form-control" type="text" name="zipcode" placeholder="Zip code.." required>
50             </div>
51             <div class="form-field">
52                 <input class="form-control" type="text" name="phone_number" placeholder="Phone Number.." required>
53             </div>
54             <div class="mt-3">
55                 <select name="payment" id="payment" class="form-control mt-3">
56                     <option value="Cash On Delivery">Cash On Delivery</option>
57                     <option value="Credit Card">Credit Card</option>
58                     <option value="Paytm">Paytm</option>
59                     <option value="Google Pay">Google Pay</option>
60                 </select>
61             </div>
62             <div>
63                 <hr>
64                 <input type="submit" value="Place Order" id="form-button" class="btn btn-success">
65             </div>
66         </form>
67     </div>
68 <br>
69 </div>
70 </div>
71 {% endblock %}
```

```
Terminal  Help  checkout.html - online-shopping - Visual Studio Code

> checkout.html 6 X
shop > templates > > checkout.html > div.container-fluid.row > div.col-lg-12 > div.box-element > div.cart-row > div > img.row-image
35     <div id="shipping-info">
36         <hr>
37         <h4>Checkout Details:</h4>
38         <hr>
39         <div class="">
40             <input class="form-control" type="text" name="address" placeholder="Address.." required>
41         </div>
42         <div class="form-field mt-3">
43             <input class="form-control" type="text" name="city" placeholder="City.." required>
44         </div>
45         <div class="form-field">
46             <input class="form-control" type="text" name="state" placeholder="State.." required>
47         </div>
48         <div class="form-field">
49             <input class="form-control" type="text" name="zipcode" placeholder="Zip code.." required>
50         </div>
51         <div class="form-field">
52             <input class="form-control" type="text" name="phone_number" placeholder="Phone Number.." required>
53         </div>
54         <div class="mt-3">
55             <select name="payment" id="payment" class="form-control mt-3">
56                 <option value="Cash On Delivery">Cash On Delivery</option>
57                 <option value="Credit Card">Credit Card</option>
58                 <option value="Paytm">Paytm</option>
59                 <option value="Google Pay">Google Pay</option>
60             </select>
61         </div>
62         <div>
63             <hr>
64             <input type="submit" value="Place Order" id="form-button" class="btn btn-success">
65         </div>
66     </div>
67 </div>
68 <br>
69 </div>
70 </div>
71 {% endblock %}
```

```
Terminal Help checkout.html - online-shopping - Visual Studio Code
< checkout.html 6 x
shop > templates > checkout.html > div.container-fluid.row > div.col-lg-12 > div.box-element > div.cart-row > div > img.row-image
66
67 </div>
68 <br>
69 </div>
70 </div>
71 {% endblock %}
72 {% block js %}
73 <script>
74     {% if alert %}
75     alert("Thankyou for placing an order. Your Order ID is {{id}}. You can use it to track your order.")
76     cart = {}
77     document.cookie = 'cart=' + JSON.stringify(cart) + ";domain=;path=/"
78     window.location.href = '/'
79     {% endif %}
80 </script>
81 {% endblock %}
```

Views.py:

```
54
55 def checkout(request):
56     data = cartData(request)
57     items = data['items']
58     order = data['order']
59     cartItems = data['cartItems']
60     total = order.get_cart_total
61     if request.method == "POST":
62         address = request.POST['address']
63         city = request.POST['city']
64         state = request.POST['state']
65         zipcode = request.POST['zipcode']
66         phone_number = request.POST['phone_number']
67         payment = request.POST['payment']
68         shipping_address = CheckoutDetail.objects.create(address=address, city=city, phone_number=phone_number, state=state, zipcode=zipcode, payment=payment)
69         shipping_address.save()
70         if total == order.get_cart_total:
71             order.complete = True
72             order.save()
73             id = order.id
74             alert = True
75             return render(request, "checkout.html", {'alert':alert, 'id':id})
76     return render(request, "checkout.html", {'items':items, 'order':order, 'cartItems':cartItems})
77
```

Code Explanation:

Then finally on the checkout page the customer can see their order summary before placing the order which is very much important. At last, the customers need to fill in the checkout details that includes the exact address, city, zip code etc and also choose the mode of payment. Then the customers can place their orders.

6. Change Password (change_password.html):

```
Terminal  Help  • change_password.html - online-shopping - Visual Studio Code

<> change_password.html 9+ •
shop > templates > <> change_password.html > script
1  {% extends 'base.html' %}
2  {% load static %}
3  {% block title %} Change Password {% endblock %}
4  {% block css %}
5  {% endblock %}
6
7  {% block body %}
8  <form class="container mt-3" method="POST" name="change_password" onsubmit="return checkPassword()">
9      {% csrf_token %}
10     <div class="row mt-4">
11         <div class="form-group col-md-6">
12             <label><i style="font-weight: bold;">Username</i></label>
13             <input type="text" class="form-control mt-2" name="username" value="{{request.user}}" readonly>
14         </div>
15         <div class="form-group col-md-6">
16             <label><i style="font-weight: bold;">Current Password</i></label>
17             <input type="password" class="form-control mt-2" name="current_password" placeholder="Current Password">
18         </div>
19     </div>
20
21     <div class="row mt-4">
22         <div class="form-group col-md-12">
23             <label><i style="font-weight: bold;">New Password</i></label>
24             <input type="password" class="form-control mt-2" name="new_password" placeholder="Enter the new password">
25         </div>
26     </div>
27
28     <div class="row mt-4">
29         <div class="form-group col-md-12">
30             <label><i style="font-weight: bold;">Confirm Password</i></label>
31             <input type="password" class="form-control mt-2" name="confirm_password" placeholder="Confirm the new password">
32         </div>
33     </div>
34
35     <input type="submit" class="btn mt-3" style="background-color: #8c5d4f; color: white;">
36 </form>
37 {% endblock %}
38
```

```
Terminal  Help  • change_password.html - online-shopping - Visual Studio Code

<> change_password.html 9+ •
shop > templates > <> change_password.html > script
34
35     <input type="submit" class="btn mt-3" style="background-color: #8c5d4f; color: white;">
36 </form>
37 {% endblock %}
38
39 {% block js %}
40 <script>
41     function checkPassword() {
42         if (document.change_password.new_password.value != document.change_password.confirm_password.value) {
43             alert("New Password and Confirm Password fields does not match each other.");
44             document.change_password.confirm_password.focus();
45             return false;
46         }
47         return true;
48     }
49
50     {% if alert %}
51     alert("Password Updated Successfully.");
52     document.location = "/logout"
53     {% endif %}
54
55     {% if currpaswrong %}
56     alert("Current Password is wrong..");
57     document.location = "/change_password"
58     {% endif %}
59 </script>
60 {% endblock %}
61
62
```

Views.py:

```
132
133 def change_password(request):
134     if not request.user.is_authenticated:
135         return redirect('/login')
136     data = cartData(request)
137     items = data['items']
138     order = data['order']
139     cartItems = data['cartItems']
140     if request.method == "POST":
141         current_password = request.POST['current_password']
142         new_password = request.POST['new_password']
143         try:
144             u = User.objects.get(id=request.user.id)
145             if u.check_password(current_password):
146                 u.set_password(new_password)
147                 u.save()
148                 alert = True
149                 return render(request, "change_password.html", {'alert':alert})
150             else:
151                 currpasswrong = True
152                 return render(request, "change_password.html", {'currpasswrong':currpasswrong})
153         except:
154             pass
155     return render(request, "change_password.html", {'cartItems':cartItems})
156
```

Code Explanation:

All the customers can change their password by going to the change password option. To check the current password (check_password) method is used and (set_password) method is used to set the new password as current password.

7. Contact Us (contact.html):

```
Terminal  Help  contact.html - online-shopping - Visual Studio Code

<> contact.html 6 X

shop > templates > <> contact.html > ...
1  {% extends 'base.html' %}
2  {% load static %}
3  {% block title %} Contact Us {% endblock %}
4  {% block css %}
5  {% endblock %}
6  {% block body %}
7  <div class="container py-2 mt-3">
8      <h2>Contact Us</h2>
9      <form method="post">{% csrf_token %}
10     <div class="form-group mt-2">
11         <label>Name</label>
12         <input type="text" class="form-control mt-2" id="name" name="name" placeholder="Enter Your Name">
13     </div>
14     <div class="form-group mt-2">
15         <label>Email</label>
16         <input type="email" class="form-control mt-2" id="email" name="email" placeholder="Enter Your Email">
17     </div>
18     <div class="form-group mt-2">
19         <label>Phone</label>
20         <input type="number" class="form-control mt-2" id="phone" name="phone" placeholder="Enter Your Phone Number">
21     </div>
22     <div class="form-group">
23         <div class="form-group mt-2">
24             <label>How May We Help You ?</label>
25             <textarea class="form-control mt-2" id="desc" name="desc" rows="3"></textarea>
26         </div>
27         <button type="submit" style="background-color: #8c5d4f; color: white; width: 8rem;" class="btn mt-4">Submit</button>
28     </form>
29 {% endblock %}
30 {% block js %}
31 <script>
32     {% if alert %}
33     alert("We will contact you soon!!!");
34     document.location = "/"
35     {% endif %}
36 </script>
37 {% endblock %}
38
```

Views.py:

```
156
157 def contact(request):
158     if request.method=="POST":
159         name = request.POST['name']
160         email = request.POST['email']
161         phone = request.POST['phone']
162         desc = request.POST['desc']
163         contact = Contact(name=name, email=email, phone=phone, desc=desc)
164         contact.save()
165         alert = True
166         return render(request, 'contact.html', {'alert':alert})
167     return render(request, "contact.html")
168
```

Code Explanation:

The customers can ask their queries or can contact us by filling a small form. There are two different forms one for the logged in users and others who haven't registered themselves but want to contact us. If the user is a logged in user, then the user just has to write the message directly else the user needs to first give his name, email and phone before contacting.

8. Track Order (tracker.html):

```
Terminal  Help  tracker.html - online-shopping - Visual Studio Code

shop > templates > tracker.html > ...
1  {% extends 'base.html' %}
2  {% block title%} Order Tracker {% endblock %}
3  {% block body %}
4  <div class="container py-5">
5      <div class="col">
6          <h2>Enter your Order Id to track your order.</h2>
7          <form method="post">{% csrf_token %}
8              <div class="form-row">
9                  <div class="form-group col-md-12">
10                     <input type="number" class="form-control mt-3" name="order_id" placeholder="Order Id">
11                 </div>
12                 <button type="submit" class="btn btn-primary mt-4">Track Order</button>
13             </div>
14         </div>
15         <div class="col my-4">
16             <h4>Order Items:</h4>
17             <div class="my-4">
18                 {% for i in order_items %}
19                 <ol class="list-group">
20                     <li class="list-group-item d-flex justify-content-between ">
21                         <div class="ms-2 me-auto">
22                             <div class="fw-bold">{{forloop.counter}}. {{i.product}}</div>
23                         </div>
24                         <span class="badge bg-primary rounded-pill">Qty: {{i.quantity}}</span>
25                     </li>
26                 </ol>
27                 {% endfor %}
28             </div>
29             <h4>Your Order Details:</h4>
30             <div class="my-4">
31                 {% for i in update_order %}
32                 <ol class="list-group">
33                     <li class="list-group-item d-flex justify-content-between ">
34                         <div class="ms-2 me-auto">
35                             <div class="fw-bold">{{i.desc}}</div>
36                         </div>
37                         <span class="badge bg-primary rounded-pill">Date: {{i.date}}</span>
38                     </li>
39                 </ol>
40                 {% endfor %}
41             </div>
42         </div>
43     </div>
44 {% endblock %}
```

```
Terminal  Help  tracker.html - online-shopping - Visual Studio Code

shop > templates > tracker.html > ...
27     {% endfor %}
28
29 </div>
30 <h4>Your Order Details:</h4>
31 <div class="my-4">
32     {% for i in update_order %}
33     <ol class="list-group">
34         <li class="list-group-item d-flex justify-content-between ">
35             <div class="ms-2 me-auto">
36                 <div class="fw-bold">{{i.desc}}</div>
37             </div>
38             <span class="badge bg-primary rounded-pill">Date: {{i.date}}</span>
39         </li>
40     </ol>
41     {% endfor %}
42 </div>
43 </div>
44 {% endblock %}
```


Views.py:

```
184
185 def tracker(request):
186     if not request.user.is_authenticated:
187         return redirect('/login')
188     data = cartData(request)
189     items = data['items']
190     order = data['order']
191     cartItems = data['cartItems']
192     if request.method == "POST":
193         order_id = request.POST['order_id']
194         order = Order.objects.filter(id=order_id).first()
195         order_items = OrderItem.objects.filter(order=order)
196         update_order = UpdateOrder.objects.filter(order_id=order_id)
197         print(update_order)
198         return render(request, "tracker.html", {'order_items':order_items, 'update_order':update_order})
199     return render(request, "tracker.html", {'cartItems':cartItems})
200
```

Code Explanation:

After placing the required order, you will get an order id. That id can be further used for tracking the order. In the track order menu, you have to give your order id for viewing the status of the order.

Result

An Online Shopping Website is successfully made using Django framework of python and HTML, CSS.

Future Scope

- a) The need for security patches in the future so as to ensure a safer and an encrypted transfer of users' information online.
- b) The need for scalability to encompass future consumable products by consumers.
- c) The update of various third-party software used in the development of this project.